

阿里规则 DSL 介绍

设计原则

基本元素

规则 DSL 语法

规则样例 1 (Java 命令注入)

规则样例 2 (PHP 命令注入)

样例规则 3 (Go 命令注入)

设计原则

本DSL的目标是给web应用安全领域的安全专家们提供一种方便的控制白盒扫描过程的配置方法。针对web应用安全扫描特点，本DSL抽象级别特定于污染分析和状态机分析，主要包含允许安全专家们轻松创建或修改污染分析和状态机分析的概念。

- R1: DSL必须允许表达以下与web应用安全相关的概念:entrance、source、sanitizer、sink、state、transition、condition。
- R2: DSL必须允许指定source、sanitizer、sink数据流传播中相关变量的详细位置，包括调用者、参数位置、返回值等粒度。
- R3: DSL必须提供不同规则间相同配置的复用能力。
- R4: DSL必须提供在现有规则基础上快速修改部分配置扩展出新规则的能力。
- R5: DSL必须能提供目前规则格式的兼容方法，使得已有规则能够方便地自动转换为符合新语法的规则。

基本元素

本质上来说，不同的污染分析模型规则即是不同的entrance、source、sanitizer、sink配置参数，不同的状态机分析模型规则即是不同的entrance、state、transition、condition配置参数。扫描规则的核心，本质上就是一组“参数”。有些配置参数只包含一个字段（例如，控制是否将所有文件作为入口文件这样的开关配置）、有些配置参数可能包含多个字段（例如，指定特定函数会净化特定位置的实参，需要函数签名和净化实位置两个字段）。因此规则的核心内容可为如下格式：

```
1    optionname = ...;
2    optionname += ...;
3    optionname.optionname = ...;
4    optionname.optionname += ...;
5    optionname.optionname = {
6        suboptionname = ...;
7        suboptionname = ...;
8    };
9    optionname.optionname += {
10        suboptionname = ...;
11        suboptionname = ...;
12    };
13
```

其中=代表新建（删除已有）一个名为"optionname"的配置参数。+=代表添加（和已有参数求并）一个名为"optionname"的配置参数（并不是所有配置都支持+=，比如说一些开关配置和缺陷类型type、缺陷子类型subtype）。delete代表删除名为"optionname"的配置参数。只包含一个字段的参数使用optionname = xxx;的形式配置。包含多个字段的参数使用optionname = {};的形式配置。这和当前的xml规则从理论上来说是一致的，单字段配置为attribute，多字段配置为property。

规则 DSL 语法

```

1  ▾ 词法规则:
2  ▾ <DEFAULT> SKIP : {
3      " "
4      | "\t"
5      | "\n"
6      | "\r"
7      | "\f"
8      }
9
10 ▾ <DEFAULT> SPECIAL : {
11 ▾ <SINGLE_LINE_COMMENT: "//" (~["\n","\r"])* ("\n" | "\r" | "\r\n")?>
12     }
13
14 ▾ <DEFAULT> MORE : {
15 ▾ "<![CDATA[" : IN_USERDEFINE_CDATA
16     | "/*" : IN_MULTI_LINE_COMMENT
17     }
18
19 ▾ <IN_USERDEFINE_CDATA> SPECIAL : {
20     <USERDEFINE_CDATA: "]]>" : DEFAULT
21     }
22
23 ▾ <IN_MULTI_LINE_COMMENT> SPECIAL : {
24     <MULTI_LINE_COMMENT: "*/"> : DEFAULT
25     }
26
27 ▾ <IN_USERDEFINE_CDATA, IN_MULTI_LINE_COMMENT> MORE : {
28     <~[]>
29     }
30
31 ▾ <DEFAULT> TOKEN : {
32 ▾ <STRING_LITERAL: "\"\" (~["\"","\\","\\n","\\r"] | "\\\" ([
33     "n","t","b","r","f","\\\\","\\'","\\\""] | ["0"-"7"] ([["0"-"7"])? | ["0"-"3"] [
34     "0"-"7"] ["0"-"7"]])* "\"">
35     }
36
37 ▾ <DEFAULT> TOKEN : {
38     <LPAREN: "(">
39     | <RPAREN: ")">
40     | <LBRACE: "{">
41     | <RBRACE: "}">
42     | <SEMICOLON: ";">
43     | <COMMA: ",">
44     | <ASSIGN: "=">
45     | <PLUSASSIGN: "+=">

```

```

44 | <PLUS: "+">
45 }
46
47 ▾ <DEFAULT> TOKEN : {
48   <RULE: "Rule">
49   | <CLASS: "Class">
50   | <IMPORT: "import">
51   | <CONST: "const">
52   | <EXTENDS: "extends">
53   | <EXCLUDE: "exclude">
54   | <TRUE: "true">
55   | <FALSE: "false">
56   | <DELETE: "delete">
57 }
58
59 ▾ <DEFAULT> TOKEN : {
60   <IDENTIFIER: <LETTER> (<LETTER> | <DIGIT>)*>
61   | <#LETTER: [" ", "a"-"z", "A"-"Z", "$"]>
62   | <#DIGIT: ["0"-"9"]>
63   | <DECIMAL_LITERAL: ["1"-"9"] (<#DIGIT>)*>
64 }
65
66 ▾ 语法规则:
67 RuleScript ::= ( ConstDeclaration )* RuleDeclaration ( UserDefineDeclaration )* <EOF>
68 ConstDeclaration ::= "const" <IDENTIFIER> "=" StringValue ";"
69 UserDefineDeclaration ::= "Class" Name ";"
70 Name ::= <IDENTIFIER> ( "." <IDENTIFIER> )*
71 ▾ RuleDeclaration ::= "Rule" <IDENTIFIER> "(" <IDENTIFIER> "," <IDENTIFIER>
72   "," <IDENTIFIER> ")" ( "extends" <IDENTIFIER> )? "{" RuleBody "}"
73 RuleBody ::= ( ImportDeclaration )* ConfigurationList
74 ImportDeclaration ::= "import" <IDENTIFIER> ( "exclude" <IDENTIFIER> (
75   "," <IDENTIFIER> )* )? ";"
76 ConfigurationList ::= ( Configuration )*
77 Configuration ::= ( ConstDeclaration | <IDENTIFIER> ( "=" | "+=" ) ( Right
78   Value | "{" SubConfigurationList "}" ) ";" | "delete" <IDENTIFIER> ";" )
79 RightValue ::= ( <DECIMAL_LITERAL> | <TRUE> | <FALSE> | StringValue )
80 StringValue ::= <STRING_LITERAL> ( "+" <STRING_LITERAL> )*
81 SubConfigurationList ::= ( <IDENTIFIER> "=" RightValue ";" )+

```

总之，我理解的web应用安全白盒扫描脚本就是：分析模型+一组配置参数。分析模型由不同的根脚本决定。计划使用javacc实现对应的词法、语法和语义分析。

规则样例 1 (Java 命令注入)

```
1 RuleCommandInjection extends AbstractTaintRule{
2     type="CommandInjection";
3     subType="commandInjectionJava";
4
5     source.method_return+={
6         precise=false;
7         value="\bParameterParser\.(getString|toQueryString)\b|(Request|
RequestWrapper)\.(getInputStream|getParameterMap|getParameterNames|getPar
parameterValues|getParameterValue|getParameter|getParameters)\b|org.apache.c
ommons.fileupload.\w+?.parseRequest\b";
8     };
9     //特定注解修饰的单个函数参数为source
10    source.param_annotation+={
11        precise=false;
12        value="org.springframework.web.bind.annotation.RequestParam";
13    };
14    //特定注解修饰函数的所有参数为source
15    source.method_annotation+={
16        precise=false;
17        value="\bDeleteMapping|\bGetMapping|\bPatchMapping|\bPostMapping|\bPutMapping|\bRequestMapping";
18    };
19    //满足特定xpath条件的函数参数为source
20    source.method_param+={
21        xpath=".//self::FormalParameter[../..[@Public='true' and @Static
='false' and @MethodName='process'] and ancestor::ClassOrInterfaceDeclarat
ion[isExtends(@ClassName, 'com.aliyun.odps.udf.xxxxxx')]]";
22        tag="UDTF";
23    };
24    source.method_param+={
25        xpath=".//self::FormalParameter[@ChildIndex = 1 and ../..[@Publi
c='true' and @Static='false' and @MethodName='iterate'] and ancestor::Clas
sOrInterfaceDeclaration[isExtends(@ClassName, 'com.aliyun.odps.xxxxxx')]]";
26        tag="UDAF";
27    };
28 }
```

规则样例 2 (PHP 命令注入)

```
1 Rule PHPCommandInjection extends AbstractTaintRule {
2     type = "PHPCommandInjection";
3     subType = "PHPCommandInjection";
4     general.desc = "PHP命令注入";
5     general.taintShellExecEcho = true;
6     source.functionReturn += "/Yii.app.request.get|Yii.app.request.post|th
ink.Request.param/";
7     source.expression += "/_GET|_POST|_COOKIE|_REQUEST/";
8     sanitizer.functionArg += {
9         pattern = "/is_numeric|is_int|is_float|is_double|ctype_digit|filte
r_var|is_bool/";
10    };
11    sanitizer.functionReturn += {
12        pattern = "/intval|floatval|boolval|oss|query|md5|OSS|oss|settype|
filter_var|preg_replace|in_array|preg_match|htmlentities|htmlspecialchars|
escapeshellcmd|escapeShellCmd|escapeshellarg/";
13    };
14    sink.functionArg += {
15        pattern = "/^exec$|^system$|shell_exec|^passthru|^popen|proc_open|
pcntl_exec/";
16        paramIndex = 0;
17    };
18 }
```

样例规则 3 (Go 命令注入)

go规则修改自 CodeQL 的 Go 命令注入规则，部分字段完全参考codeql实现

```

1 Rule Go_CommandInjection extends AbstractTaintRule {
2     source.expression += "net/http\\.Request\\. (Body|PostForm|Form|GetBody
|MultipartForm|Header|Trailer)";
3     source.functionReturn += "net/url\\.Values\\.Get|net/http\\.Request\\.
FormValue";
4     source.functionArg += {
5         pattern = "(github\\.com/astaxie/beego/context|github\\.com/beego/
beego/context|github\\.com/beego/beego/v2/server/web/context)\\.Bind";
6         paramIndex = 0;
7     };
8     source.expression += "(github\\.com/astaxie/beego/context\\.BeegoInput
|github\\.com/beego/beego/context\\.BeegoInput|github\\.com/beego/beego/v
2/server/web/context\\.BeegoInput)\\.RequestBody";
9     source.functionReturn += "(github\\.com/astaxie/beego/context|github
\\.com/beego/beego/context|github\\.com/beego/beego/v2/server/web/context)
\\.Context\\.GetCookie";
10    ...
11    sink.functionArg += {
12        pattern = "os\\.StartProcess|os/exec\\.Command$";
13        paramIndex = 1;
14    };
15    ...
16    propagate.customFunctionPropagate += {
17        from = 0;
18        pattern = "encoding/json\\.Unmarshal";
19        to = 1;
20    };
21    propagate.customFunctionPropagate += {
22        from = -1;
23        pattern = "encoding/json\\.Decoder\\.Decode";
24        to = 0;
25    };
26    ...
27    sanitizer.functionArg += "math\\.IsInf|math\\.IsNaN";
28    sanitizer.functionReturn += {
29        pattern = "strconv\\.ParseInt|strconv\\.Atoi|strconv\\.ParseFloat|
strconv\\.ParseBool";
30    };
31    sanitizer.functionReturn += {
32        pattern = "encoding/hex\\.EncodeToString";
33    };
34    sanitizer.functionReturn += {
35        pattern = "crypto/md5\\.Sum";
36    };
37    type = "GoCommandInjection";

```

```
38     subType = "GoCommandInjection";  
39     general.desc = "golang 命令注入";  
40 }  
41
```